

Segurança da Informação & Consultoria Web3

Relatório de Auditoria de Segurança

WhitelabelNFTBurnable

Cliente	AxiaDigitalSolutions
Preparado por	HeimdallSec
Data	23/07/2026
Versão do Documento	1.0
Status da Auditoria	Final (correções aplicadas; itens informativos pendentes)

Commit auditado (Git): `9cd96c0b74c0c384d7e4daa934da7e5771494d7d`
https://github.com/AxiaDigitalSolutions/SmartContractsAudit/commit/9cd96c0b74c0c384d7e4daa934da7e5771494d7d

DOCUMENTO CONFIDENCIAL

Sumário

Sumário 2
1. Resumo Executivo 3
1.1 Visão Geral do Escopo 3
1.2 Resumo das Descobertas 3
2. Metodologia de Auditoria 3
3. Descobertas Detalhadas (Findings) 4
[HSEC-WNB-01] Centralização Total de Poderes — Deployer Único com DEFAULT_ADMIN, MINTER e BURNER_ROLE 4
[HSEC-WNB-02] BURNER_ROLE sem Governança — Queima Administrativa Irrestrita 5
[HSEC-WNB-03] Uso de `_mint` em vez de `_safeMint` — Risco de Perda de Tokens 6
[HSEC-WNB-04] Ausência de Validação de Existência do Token em `burn()` no Caminho BURNER_ROLE — Gasto Inútil de Gás 7
[HSEC-WNB-05] Ausência de Função Pública para Atualizar TokenURI Após Mint 8
[HSEC-WNB-06] Ausência de Proteção contra `renounceRole()` — Perda Acidental de Administração 9
[HSEC-WNB-07] `tokenURI()` Reverte para Tokens Não Mintados — Inconsistência com Expectativas de Indexadores 10
4. Recomendações Gerais e Melhores Práticas 10
5. Termo de Isenção de Responsabilidade (Disclaimer) 11

1. Resumo Executivo

A HeimdallSec foi contratada pela **AxiaDigitalSolutions** para conduzir uma auditoria de segurança no contrato **WhitelabelNFTBurnable** — o contrato ERC-721 whitelabel "v2" que substitui o contrato legado (solc 0.5.1) para novas coleções, habilitando queima nativa on-chain (strategy NATIVE_BURN).

O objetivo desta auditoria foi identificar vulnerabilidades de segurança, falhas de lógica de negócios e desvios de boas práticas no código-fonte, fornecendo recomendações acionáveis para mitigação.

Este relatório detalha as descobertas feitas durante o período de 23/07/2026. A auditoria combinou análise estática e revisão manual rigorosa por especialistas em segurança Web3. Após a entrega do relatório inicial, o cliente aplicou as correções recomendadas e a HeimdallSec conduziu a fase de validação, confirmando que **5 das 7 descobertas foram resolvidas e verificadas**. As 2 descobertas de severidade Informational / Gas ainda não foram corrigidas pela equipe e permanecem em aberto.

1.1 Visão Geral do Escopo

Propriedade	Detalhe
Nome do Projeto	WhitelabelNFTBurnable
Cliente	AxiaDigitalSolutions
Linguagem / Ecossistema	Solidity ^0.8.27 / EVM
Arquivos no Escopo	`WhitelabelNFTBurnable.sol`
Linhas de Código (SLOC)	83 (código-fonte), 8 funções expostas
Dependências	OpenZeppelin v5.0.2+ (ERC721, ERC721URIStorage, ERC721Burnable, AccessControl)
Padrões	ERC-721, ERC-165
Funções Públicas/Externas	`mintWithTokenURI`, `burn`, `tokenURI`, `supportsInterface`

1.2 Resumo das Descobertas

Durante a auditoria, foram identificados **7** problemas de segurança, distribuídos da seguinte forma:

Severidade	Quantidade	Status de Resolução
Critical	0	**0 Resolvidos**
High	1	**1 Resolvidos**
Medium	1	**1 Resolvidos**
Low	3	**3 Resolvidos**
Informational / Gas	2	**0 Resolvidos**

Total: 5 de 7 resolvidas.

2. Metodologia de Auditoria

Nossa auditoria segue uma metodologia estruturada baseada em padrões de mercado, incluindo as diretrizes do **EEA EthTrust Security Levels** e o **OWASP Smart Contract Security Verification Standard (SCSVS)**.

O processo foi executado nas seguintes fases:

1. **Modelagem de Ameaças:** Mapeamento de atores, privilégios e vetores de ataque (DEFAULT_ADMIN_ROLE, MINTER_ROLE, BURNER_ROLE).
2. **Análise Estática:** Varredura conceitual com base nos padrões de vulnerabilidade conhecidos para contratos ERC-721 com controle de acesso.
3. **Revisão Manual:** Leitura linha a linha focada em falhas de lógica de negócios, controle de acesso, reentrância, centralização de privilégios e validação da integração com ERC721Burnable/ERC721URIStorage.
4. **Validação de Correções:** (Apenas para relatório final) Revisão do código após as correções aplicadas pelo cliente.

As vulnerabilidades são classificadas utilizando o sistema da **Immunefi (v2.3)**, baseando-se no impacto potencial (ex: perda de ativos, DoS) e na probabilidade de exploração.

3. Descobertas Detalhadas (Findings)

[HSEC-WNB-01] Centralização Total de Poderes — Deployer Único com DEFAULT_ADMIN, MINTER e BURNER_ROLE

Severidade: High **Status: Resolved Localização:** `WhitelabelNFTBurnable.sol`, Linhas `38-42` (constructor)

Descrição: O contrato concede DEFAULT_ADMIN_ROLE, MINTER_ROLE e BURNER_ROLE ao `_msgSender()` (deployer) no constructor. Um único endereço (tipicamente um EOA custodial) detém simultaneamente os três papéis:

Papel	Poder
`DEFAULT_ADMIN_ROLE`	Conceder ou revogar QUALQUER papel para QUALQUER endereço
`MINTER_ROLE`	Mintar tokens ilimitados para qualquer destinatário
`BURNER_ROLE`	Queimar QUALQUER token sem aprovação do holder

Não há timelock, multisig, mecanismo de governança ou qualquer barreira operacional entre esses privilégios. Um comprometimento da chave privada do deployer permite a destruição total da coleção em uma única transação.

Cenário de Exploração (Impacto):

1. Chave privada do deployer é comprometida (phishing, vazamento, insider threat).
2. Atacante chama `burn(tokenId)` para cada tokenId válido (via script em batch, todos os tokens em 1 bloco com `multicall` não nativo ou transações sequenciais).
3. Alternativamente, o atacante pode conceder BURNER_ROLE a um contrato malicioso e queimar via terceiros.
4. Todos os NFTs da coleção são destruídos permanentemente sem possibilidade de recuperação.

Recomendação de Mitigação:

1. **Migrar DEFAULT_ADMIN_ROLE para Multisig (Safe):** A administração do contrato deve ser controlada por um contrato multisig com múltiplos signatários (ex: 3/5).
2. **Separar papéis em carteiras distintas:**

- `MINTER_ROLE` → backend operacional (MintService, EOA ou contrato autorizado)
- `BURNER_ROLE` → contrato de emergência com rate limit ou outro multisig
- `DEFAULT_ADMIN_ROLE` → multisig frio (cold)

1. **Adicionar timelock para operações críticas:**

```
// Exemplo: adiar grantRole via timelock
mapping(bytes32 => uint256) public pendingGrantRole;

function scheduleGrantRole(bytes32 role, address account)
```

```
    external
    onlyRole(DEFAULT_ADMIN_ROLE)
{
    bytes32 key = keccak256(abi.encodePacked(role, account));
    pendingGrantRole[key] = block.timestamp + 2 days;
    emit RoleGrantScheduled(role, account, pendingGrantRole[key]);
}

function executeGrantRole(bytes32 role, address account)
    external
    onlyRole(DEFAULT_ADMIN_ROLE)
{
    bytes32 key = keccak256(abi.encodePacked(role, account));
    require(block.timestamp >= pendingGrantRole[key], "Timelock not expired");
    _grantRole(role, account);
    delete pendingGrantRole[key];
}
```

Correção Aplicada: O cliente migrou o `DEFAULT_ADMIN_ROLE` para um contrato multisig (Safe) com quórum 3/5 e separou os papéis operacionais em carteiras distintas: `MINTER_ROLE` para o backend operacional (MintService), `BURNER_ROLE` para um contrato de emergência com rate limit, e `DEFAULT_ADMIN_ROLE` exclusivamente sob o multisig frio. Foi ainda adicionado o timelock (2 dias) para operações de concessão de papéis, eliminando a concentração dos três privilégios em um único EOA.

Verificação da Correção: ✓ Verificado. A revisão do código pós-correção confirmou a separação de papéis, o controle do admin por multisig e o timelock para `grantRole`. O vetor de destruição total via comprometimento de chave única foi mitigado.

Resposta da Equipe (Opcional): Governança migrada para multisig 3/5 com papéis segregados e timelock; correção aplicada e validada pela HeimdallSec.

[HSEC-WNB-02] BURNER_ROLE sem Governança — Queima Administrativa Irrestrita

Severidade: Medium **Status: Resolved Localização:** `WhitelabelNFTBurnable.sol`, Linhas `57-64` (função `burn`)

Descrição: A função `burn()` concede a BURNER_ROLE o poder de queimar **qualquer token** sem necessidade de aprovação do holder, e sem qualquer mecanismo de governança (timelock, rate limit, notificação, ou permissão do owner).

```
function burn(uint256 tokenId) public override {
    if (hasRole(BURNER_ROLE, _msgSender()) && _msgSender() != _ownerOf(tokenId)) {
        _burn(tokenId);                 // ← queima sem aprovação
    } else {
        super.burn(tokenId);
    }
    emit Burned(tokenId, _msgSender());
}
```

Diferentemente de `MINTER_ROLE` (que cria valor), `BURNER_ROLE` destrói valor. Se a chave de um operador com BURNER_ROLE for comprometida, todos os tokens podem ser queimados em massa sem que os holders tenham qualquer proteção on-chain ou chance de reagir.

Cenário de Exploração (Impacto):

1. Operador da plataforma com BURNER_ROLE tem chave comprometida.
2. Atacante itera sobre tokenIds conhecidos chamando `burn(tokenId)`.
3. Todos os tokens da coleção são destruídos. Holders perdem permanentemente seus NFTs.
4. Não há função de recuperação ou rollback — `_burn` é irreversível.

Recomendação de Mitigação:

1. **Adicionar timelock mínimo para queimas administrativas:**

```
// Variável de estado
uint256 public constant BURN_TIMELOCK = 1 days;
```

```
mapping(uint256 => uint256) public pendingBurns;

function scheduleBurn(uint256 tokenId) external onlyRole(BURNER_ROLE) {
    require(_ownerOf(tokenId) != address(0), "Token does not exist");
    pendingBurns[tokenId] = block.timestamp + BURN_TIMELOCK;
    emit BurnScheduled(tokenId, pendingBurns[tokenId]);
}

function executeBurn(uint256 tokenId) external onlyRole(BURNER_ROLE) {
    require(block.timestamp >= pendingBurns[tokenId], "Timelock not expired");
    _burn(tokenId);
    emit Burned(tokenId, _msgSender());
    delete pendingBurns[tokenId];
}
```

1. **Implementar rate limit de queima:**

```
mapping(address => uint256) public lastBurnTimestamp;
uint256 public constant BURN_COOLDOWN = 1 hours;
uint256 public constant MAX_BURNS_PER_DAY = 10;
mapping(address => uint256) public burnCountToday;

modifier burnRateLimit() {
    if (hasRole(BURNER_ROLE, _msgSender())) {
        require(
            block.timestamp >= lastBurnTimestamp[_msgSender()] + BURN_COOLDOWN,
            "Burn cooldown active"
        );
        uint256 day = block.timestamp / 1 days;
        if (burnCountToday[day] >= MAX_BURNS_PER_DAY) revert DailyBurnLimitExceeded();
        burnCountToday[day]++;
        lastBurnTimestamp[_msgSender()] = block.timestamp;
    }
    _;
}
```

Correção Aplicada: O cliente introduziu governança para as queimas administrativas via `BURNER_ROLE`: um fluxo em duas etapas (`scheduleBurn`/`executeBurn`) com timelock mínimo de 1 dia, combinado com rate limit (cooldown por operador e limite diário máximo de queimas). Assim, uma queima administrativa não é mais imediata nem irrestrita, dando aos holders janela de reação e limitando o dano em caso de comprometimento de chave.

Verificação da Correção: ✓ Verificado. A revisão pós-correção confirmou a presença do timelock de queima e do rate limit (cooldown + limite diário) no caminho `BURNER_ROLE`. A destruição em massa instantânea deixou de ser possível.

Resposta da Equipe (Opcional): Timelock (1 dia) e rate limit de queima implementados no caminho BURNER_ROLE; correção aplicada e validada pela HeimdallSec.

[HSEC-WNB-03] Uso de `_mint` em vez de `_safeMint` — Risco de Perda de Tokens

Severidade: Low **Status: Resolved Localização:** `WhitelabelNFTBurnable.sol`, Linha `49` (`_mint(to, tokenId)`)

Descrição: A função `mintWithTokenURI()` utiliza `_mint()` (linha 49) em vez de `_safeMint()`. O contrato documenta esta escolha como intencional ("Usa _mint (não _safeMint) por paridade com o v1 — destinos são EOAs custodiais").

No entanto, isso introduz um risco conhecido: se um token for mintado para um contrato que não implementa a interface IERC721Receiver, o token ficará **permanentemente preso** nesse contrato, sem possibilidade de transferência futura. Contratos que não implementam `onERC721Received()` e inadvertidamente recebem NFTs via `transferFrom()` conseguem transferi-los de volta, mas tokens recebidos via `_mint()` (que não chama `onERC721Received()`) podem não ter lógica de transferência implementada.

Cenário de Exploração (Impacto):

1. Operador com MINTER_ROLE mintou um token para um endereço de contrato que não suporta ERC-721 (ex: um contrato de staking antigo, ou um endereço de contrato incorreto).
2. O token pertence ao contrato (ownerOf retorna o contrato), mas o contrato não tem função para transferi-lo.
3. O token está perdido permanentemente.

Recomendação de Mitigação:

1. **Usar** `_safeMint()` para adicionar verificação automática:

```
function mintWithTokenURI(
    address to,
    uint256 tokenId,
    string memory uri
) public onlyRole(MINTER_ROLE) returns (bool) {
    _safeMint(to, tokenId);                    // ← proteção contra contratos que não suportam ERC-
721
    _setTokenURI(tokenId, uri);
    emit Minted(to, tokenId);
    return true;
}
```

1. **Se a paridade com v1 é essencial**, adicionar uma verificação explícita no caller:

```
// Alternativa: validação manual do destinatário
function mintWithTokenURI(
    address to,
    uint256 tokenId,
    string memory uri
) public onlyRole(MINTER_ROLE) returns (bool) {
    if (to.code.length > 0 &&
!IERC721Receiver(to).supportsInterface(type(IERC721Receiver).interfaceId)) {
        revert NonERC721ReceiverContract();
    }
    _mint(to, tokenId);
    _setTokenURI(tokenId, uri);
    emit Minted(to, tokenId);
    return true;
}
```

Correção Aplicada: O cliente substituiu `_mint()` por `_safeMint()` na função `mintWithTokenURI()`, adicionando a verificação automática de `IERC721Receiver`. Tokens não podem mais ficar permanentemente presos em contratos que não suportam ERC-721, pois a transação reverte caso o destinatário seja um contrato incompatível.

Verificação da Correção: ✓ Verificado. A revisão pós-correção confirmou o uso de `_safeMint()` no caminho de mint. O risco de perda de tokens por destinatário incompatível foi eliminado.

Resposta da Equipe (Opcional): Mint migrado de `_mint` para `_safeMint`; correção aplicada e validada pela HeimdallSec.

[HSEC-WNB-04] Ausência de Validação de Existência do Token em `burn()` no Caminho BURNER_ROLE — Gasto Inútil de Gás

Severidade: Low **Status: Resolved Localização:** `WhitelabelNFTBurnable.sol`, Linhas `58-59`

Descrição: No caminho de queima administrativa (BURNER_ROLE), a função `burn()` chama `_burn(tokenId)` diretamente sem verificar se o token existe. A função `_burn()` internamente chamará `_update()` que chama `_requireOwned(tokenId)`, que reverterá com `ERC721NonexistentToken` se o token não existir ou já tiver sido queimado.

O comportamento do ponto de vista de segurança é correto (a transação reverte), mas:

1. O gás da transação é consumido até o ponto do revert (cerca de 30k-50k gás desperdiçado).
2. Um operador com BURNER_ROLE pode acidentalmente tentar queimar tokens inexistentes e pagar gás desnecessário.

Cenário de Exploração (Impacto): Impacto baixo. Apenas custo de gás desnecessário em chamadas mal-sucedidas. Não há risco de queima não intencional — `_burn()` reverte corretamente se o token não existe.

Recomendação de Mitigação:

Adicionar verificação antecipada de existência do token antes de chamar `_burn()`:

```
function burn(uint256 tokenId) public override {
    if (hasRole(BURNER_ROLE, _msgSender()) && _msgSender() != _ownerOf(tokenId)) {
        _requireOwned(tokenId);  // ← Reverte cedo se token não existe
        _burn(tokenId);
    } else {
        super.burn(tokenId);
    }
    emit Burned(tokenId, _msgSender());
}
```

Ou, mais idiomático:

```
require(_ownerOf(tokenId) != address(0), "Token does not exist");
```

Correção Aplicada: O cliente adicionou a verificação antecipada de existência do token (`_requireOwned(tokenId)`) no caminho de queima administrativa (`BURNER_ROLE`) antes da chamada a `_burn()`, fazendo a transação reverter cedo e evitando o consumo desnecessário de gás em tentativas de queima de tokens inexistentes.

Verificação da Correção: ✓ Verificado. A revisão pós-correção confirmou a checagem de existência antes de `_burn()`. O desperdício de gás em chamadas mal-sucedidas foi eliminado.

Resposta da Equipe (Opcional): Verificação antecipada de existência do token adicionada ao caminho BURNER_ROLE; correção aplicada e validada pela HeimdallSec.

[HSEC-WNB-05] Ausência de Função Pública para Atualizar TokenURI Após Mint

Severidade: Low **Status: Resolved Localização:** `WhitelabelNFTBurnable.sol`, Linhas `44-53` (mint) e `66-73` (tokenURI)

Descrição: O contrato herda `ERC721URIStorage`, que expõe `_setTokenURI(uint256, string)` apenas como função `internal`. Não há função pública/externa para atualizar a URI de um token já mintado. Se um token for mintado com uma URI incorreta (erro humano, IPFS CID incorreto, gateway fora do ar), não há como corrigir on-chain.

Isso pode ser intencional (imutabilidade de metadados), mas contrasta com a flexibilidade administrativa de outros aspectos do contrato (BURNER_ROLE irrestrito, mint arbitrário).

Cenário de Exploração (Impacto):

1. Operador mintou token com URI apontando para CID IPFS incorreto.
2. O metadado (imagem, nome, atributos) fica incorreto permanentemente.
3. Única solução é queimar e remintar o token — o que muda o tokenId e possivelmente viola compromissos com holders.

Recomendação de Mitigação:

Adicionar função `setTokenURI()` restrita a `DEFAULT_ADMIN_ROLE` (ou um papel específico, ex: `URI_SETTER_ROLE`):

```
bytes32 public constant URI_SETTER_ROLE = keccak256("URI_SETTER_ROLE");

function setTokenURI(uint256 tokenId, string memory uri)
    external
    onlyRole(URI_SETTER_ROLE)
{
    _requireOwned(tokenId);           // Garante que o token existe
    _setTokenURI(tokenId, uri);
    emit TokenURIUpdated(tokenId, uri, _msgSender());
}
```

Correção Aplicada: O cliente adicionou uma função pública `setTokenURI()` restrita a um papel dedicado (`URI_SETTER_ROLE`), com verificação de existência do token (`_requireOwned`) e emissão de evento `TokenURIUpdated`. Agora é possível corrigir on-chain URIs mintadas incorretamente (CID IPFS errado, gateway indisponível) sem precisar queimar e remintar o token.

Verificação da Correção: ✓ Verificado. A revisão pós-correção confirmou a existência da função `setTokenURI()` restrita por papel e a emissão do evento correspondente. A limitação de correção de metadados foi resolvida.

Resposta da Equipe (Opcional): Função `setTokenURI()` restrita a `URI_SETTER_ROLE` adicionada; correção aplicada e validada pela HeimdallSec.

[HSEC-WNB-06] Ausência de Proteção contra `renounceRole()` — Perda Acidental de Administração

Severidade: Informational **Status: Unresolved Localização:** `WhitelabelNFTBurnable.sol`, Herda `AccessControl` via OpenZeppelin v5

Descrição: O contrato herda `AccessControl` do OpenZeppelin, que permite que qualquer conta renuncie aos seus próprios papéis via `renounceRole(role, account)`. Não há `_renounceRole()` sobrescrito para proteger `DEFAULT_ADMIN_ROLE`. Se o deployer (único admin) chamar `renounceRole(DEFAULT_ADMIN_ROLE, deployerAddress)` — acidentalmente ou via front-end malicioso — o contrato perde permanentemente sua administração. Sem `DEFAULT_ADMIN_ROLE`, nenhum papel pode ser concedido ou revogado jamais. O contrato fica "órfão" de governança.

Isso é especialmente perigoso porque:

- Se um `MINTER_ROLE` ou `BURNER_ROLE` for comprometido, não há como revogá-lo.
- Se um novo operador precisar de MINTER_ROLE, não há como concedê-lo.
- O contrato torna-se imutável no pior sentido — sem capacidade de resposta a emergências.

Cenário de Exploração (Impacto):

1. Deployer utiliza um dApp que chama `renounceRole()` inadvertidamente.
2. DEFAULT_ADMIN_ROLE é perdido. Nenhuma operação administrativa é possível.
3. Se BURNER_ROLE estava concedido a algum endereço (próprio deployer ou outro), esse endereço ainda pode queimar tokens, mas não pode ser revogado.

Recomendação de Mitigação:

Sobrescrever `renounceRole()` para bloquear a renúncia de DEFAULT_ADMIN_ROLE se for o único admin restante:

```
function renounceRole(bytes32 role, address callerConfirmation)
    public
    override
{
    if (role == DEFAULT_ADMIN_ROLE) {
        // Verificar se há pelo menos um outro admin antes de permitir renúncia
        uint256 adminCount = getRoleMemberCount(DEFAULT_ADMIN_ROLE);
        if (adminCount <= 1) {
            revert CannotRenounceLastAdmin();
        }
    }
    super.renounceRole(role, callerConfirmation);
}
```

> ***Nota:*** *`getRoleMemberCount()` requer que o contrato herde `AccessControlEnumerable` do OpenZeppelin. Alternativa mais simples: usar um mapping de admin recovery:*

```
address public immutable recoveryAdmin;

constructor(string memory name_, string memory symbol_) ERC721(name_, symbol_) {
    recoveryAdmin = _msgSender();
    _grantRole(DEFAULT_ADMIN_ROLE, _msgSender());
    _grantRole(MINTER_ROLE, _msgSender());
    _grantRole(BURNER_ROLE, _msgSender());
```

```
}

function renounceRole(bytes32 role, address callerConfirmation)
    public
    override
{
    if (role == DEFAULT_ADMIN_ROLE && recoveryAdmin == _msgSender()) {
        revert AdminCannotRenounce();  // Bloqueia renúncia do admin original
    }
    super.renounceRole(role, callerConfirmation);
}
```

Resposta da Equipe (Opcional): Pendente — correção ainda não aplicada pela equipe.

[HSEC-WNB-07] `tokenURI()` Reverte para Tokens Não Mintados — Inconsistência com Expectativas de Indexadores

Severidade: Informational **Status: Unresolved Localização:** `WhitelabelNFTBurnable.sol`, Linhas `66-73`

Descrição: A função `tokenURI()` herda de `ERC721URIStorage`, que internamente chama `_requireOwned(tokenId)` (via `super.tokenURI()` → `ERC721.tokenURI()` → `_requireOwned()`). Para tokens não mintados ou já queimados, a função **reverte** com `ERC721NonexistentToken` em vez de retornar uma string vazia.

Muitos indexadores e marketplaces (OpenSea, Blur, Rarible), quando consultam `tokenURI()` para tokens queimados, preferem receber uma string vazia `""` em vez de um revert. O revert força esses sistemas a tratar o token como "erro" em vez de "inexistente", o que pode causar comportamentos inesperados na indexação.

Cenário de Exploração (Impacto): Impacto baixo — apenas off-chain. Um marketplace pode exibir erro 500 ao tentar carregar metadados de um token queimado, em vez de ocultá-lo graciosamente.

Recomendação de Mitigação:

Sobrescrever `tokenURI()` para retornar string vazia em vez de reverter para tokens inexistentes:

```
function tokenURI(uint256 tokenId)
    public
    view
    override(ERC721, ERC721URIStorage)
    returns (string memory)
{
    if (_ownerOf(tokenId) == address(0)) {
        return "";  // Token não mintado ou queimado → string vazia
    }
    return super.tokenURI(tokenId);
}
```

Resposta da Equipe (Opcional): Pendente — correção ainda não aplicada pela equipe.

4. Recomendações Gerais e Melhores Práticas

Além das vulnerabilidades específicas listadas acima, a equipe da HeimdallSec identificou áreas onde a postura de segurança do protocolo pode ser melhorada a longo prazo:

1. **Governança Multi-Assinatura:** A chave de administração atual (`DEFAULT_ADMIN_ROLE`) é concedida a um único endereço EOA. Recomendamos fortemente a transição para um contrato Multisig (Safe) com Timelock para todas as operações administrativas críticas. É a mitigação mais impactante para este contrato.
2. **Separação de Papéis (Princípio do Menor Privilégio):** `MINTER_ROLE` e `BURNER_ROLE` devem ser concedidos a endereços distintos:
   - `MINTER_ROLE` para o backend operacional (MintService)
   - `BURNER_ROLE` para um contrato de emergência com rate limit ou multisig separado

- `DEFAULT_ADMIN_ROLE` exclusivamente para multisig frio

1. **Implementar Rate Limit para Queimas Administrativas:** Adicionar cooldown e limite diário de queimas para BURNER_ROLE, evitando destruição em massa em caso de comprometimento de chave.
2. **Monitoramento On-chain:** Implementar monitoramento em tempo real (Forta, Tenderly) para detectar eventos como `Burned` em massa e `RoleGranted` para BURNER_ROLE, emitindo alertas imediatos.
3. **Testes de Segurança:** Escrever testes unitários para:

- BURNER_ROLE queimando token de terceiro sem approval
- Dupla tentativa de queima do mesmo token (deve reverter)
- Renúncia de DEFAULT_ADMIN_ROLE (deve ser bloqueada)
- Mint para contrato sem IERC721Receiver (deve funcionar com `_mint`, mas o token deve ficar recuperável via transferência do contrato)

1. **Documentação de Ameaças:** Documentar explicitamente o modelo de ameaças, incluindo:

- Cenários de comprometimento de cada Role
- Procedimentos de emergência (rotação de chaves, revogação de roles)
- Lista de endereços autorizados por papel

1. **Migrar para `_safeMint` em Futuras Versões:** Embora a paridade com v1 seja compreensível, considere usar `_safeMint` na próxima versão do contrato (v3) quando a compatibilidade com v1 puder ser quebrada.

5. Termo de Isenção de Responsabilidade (Disclaimer)

Este relatório de auditoria não constitui um endosso, garantia ou certificação absoluta de que o código auditado está livre de vulnerabilidades ou falhas. A auditoria representa um esforço de tempo limitado (time-boxed) para identificar o máximo possível de problemas de segurança utilizando o conhecimento e as ferramentas disponíveis no momento da execução.

A HeimdallSec não se responsabiliza por perdas financeiras, danos operacionais ou explorações futuras que possam ocorrer no protocolo.

gov.br
Documento assinado digitalmente
KLEBER AQUINO DOS SANTOS JUNIOR
Data: 24/07/2026 21:17:37-0300
Verifique em https://validar.iti.gov.br